# Contents

**Demand estimation: from purchase histories to a simulated likelihood** **2**
Contents . . . 2
1. What is being estimated? . . . 2
2. Source map and control flow . . . 2
3. Turning purchase lines into weekly choice observations . . . 3
4. The history index, derived and worked through . . . 4
5. Utility and the four specifications . . . 5
6. The likelihood: why integration comes after summing the history . . . 6
7. The analytic gradient and numerical optimization . . . 6
8. The price control, step by step . . . 7
9. Household bootstrap and its scope . . . 8
10. Running and reading the results . . . 8

# Demand estimation: from purchase histories to a simulated likelihood

This chapter explains `est.py`, its four entry points, `price_control.py`, and `bootstrap_se.py`. It follows the current source rather than treating older README paragraphs as authoritative. For the upstream construction, see the data chapter; for the separate theoretical pricing exercise, see the supply chapter.

## Contents


1. Economic question and observation units
2. Source map and execution order
3. Purchase records, weekly counts, and menus
4. History construction and a worked example
5. Utility, normalizations, and heterogeneity
6. Household likelihood and stable integration
7. Analytic score and optimization
8. Other-DMA instruments and residual controls
9. Household bootstrap
10. Commands, outputs, diagnosis, and extensions


## 1. What is being estimated?

The baseline question is: conditional on a household buying yogurt in an observed week, how do its flavor purchases depend on prices and its past flavor purchases? The alternatives, in fixed array order, are `other`, `berry`, and `plain`. They are groups of UPCs, not brands or individual products.

There are four distinct units. An input row is a purchase line. Prices originate as store/week/UPC quotes. The choice observation is a household-week count vector. The unit over which persistent random preferences are integrated and resampled is the household. Mixing these units up changes the estimator.

Write household as $i$, calendar week as $t$, and flavor as $j$. The observed count $q_{ijt}$ counts packages, and $Q_{it} = \sum_j q_{ijt}$. A week buying two berry packages and one plain package is one row with counts $(0, 2, 1)$, not a categorical label of "berry." The model conditions on the observed number of packages. It does not explain the decision to buy three packages rather than one.

By default `capacity=0`: there is no outside alternative. Missing purchase weeks are not inserted. This is central to interpretation: category participation, total yogurt demand, and substitution into non-yogurt goods are not estimated.

## 2. Source map and control flow

| Source | Main responsibility |
|---|---|
| `est.py` | Input preparation, histories, arrays, draws, utility, likelihood, score, optimization, output |
| `est_naive.py` | Calls `main` with model `naive` |
| `est_gamma.py` | Calls `main` with model `homogeneous` |
| `est_het_naive.py` | Calls `main` with model `mixed_naive` |
| `est_het_gamma.py` | Calls `main` with model `heterogeneous` |
| `price_control.py` | DMA validation, fixed-effect absorption, flavor first stages |
| `bootstrap_se.py` | Whole-household resampling, repeated two-stage fitting, inference |
| `test_est.py` | Synthetic checks of histories, preparation, likelihood, gradients, and related behavior |
| `test_price_control.py` | Synthetic price-control and bootstrap checks |
| `test_summary_stats.py` | Checks of upstream summary behavior |

`main()` parses and validates arguments, calls `prepare_data`, optionally calls `add_price_controls`, saves weekly inputs, creates numerical arrays, and calls `fit_model`. The latter calls `optimize_demand`, saves point estimates and the gamma distribution, runs inference if appropriate, and writes final output.

The scripts import sibling modules directly. Run commands from this directory or invoke the script by its path. The code is not installed as a Python package. Importing `est.py` does not read the private input data.

# 3. Turning purchase lines into weekly choice observations

## 3.1 Inputs and initial checks

The default prepared input is the file `yogurt_estimation_sample.parquet` under the following directory:

```
/scratch/dtm63837/Kilts_Panel/Data/summary_outputs/
```

Its important fields are household ID, panel year, purchase date, trip ID, UPC, store ID, positive integer quantity, flavor, and merged price. With price controls, the consumer DMA is required as well. `--household-dma-column` changes its source name. `price` takes precedence over `price_rms`.

`prepare_data(args)` scans with Polars, restricts panel years, normalizes UPCs, and uses saved flavor labels if present. Only older inputs without a `flavor` column require `read_flavors`. Saved labels outside the three groups stop the run. Invalid/nonpositive prices are filtered before weekly histories are built. Quantities in the loaded demand sample must be finite, positive, and integer-valued.

UPC normalization is string based: `"000123.0"` becomes `"123"`. Avoiding a floating-point conversion protects long identifiers. `read_flavors` can read an already resolved `upc,flavor` mapping, which must have unique normalized UPCs, or classify descriptive text with plain precedence over berry terms. Its legacy recovery path supplements absent UPCs from product metadata.

The development option `--max-households` takes the smallest household IDs. It is deterministic, not a random subsample. The quote universe still comes from the full year-filtered input, so limiting demand households does not deliberately shrink their menus or the instrument universe.

## 3.2 Calendar time

The week ends on Saturday. Polars weekdays are Monday=1 through Sunday=7, so the current offset is $(6 - \text{weekday}) \bmod 7$. Saturday has offset zero and Sunday offset six. A Sunday purchase therefore belongs to the following Saturday, not the preceding one.

The estimator recomputes weeks from purchase dates. The checked-in upstream `panel_week.py` formula agrees with it under Polars' 1–7 weekday convention. Older notes warn of Sunday losses; that warning requires checking the historical code/data version rather than assuming the current formula is wrong. If an earlier build did use

incorrect weeks, recomputation cannot restore purchases already discarded by its inner join. The data chapter works through both formulas.

### 3.3 Counts and exposure stores

Package counts sum purchases across **all stores** in a household-week. The exposure store is the store at which the household bought the most packages that week; a tie goes to the lowest store code. All alternatives are priced using that exposure store's menu. Counts and prices consequently have different aggregation rules. A purchase at a second store still contributes to the count.

For prices, duplicate store/week/UPC quotes are removed before averaging. A store/week/flavor price is the arithmetic mean of its observed UPC quotes, with one weight per UPC quote. It is not expenditure per package, a quantity-weighted average, or an average of the household's purchases. No `prmult` adjustment is made. Conflicting price or flavor values for the same quote key stop preparation.

Because quotes are recovered from merged purchase records, a price absent here does not establish that the store did not offer the product. The menu is a maintained measurement rule based on available records.

### 3.4 Availability and the order of exclusions

A finite positive flavor price makes that alternative available. An unpriced flavor with zero purchases is simply excluded from that week's denominator. An unpriced flavor with positive purchases invalidates the week. With controls, a missing other-DMA instrument also invalidates the likelihood observation.

The order is deliberate:

1. Form weekly counts and attach menus/instruments.
2. Build histories from all these observed positive-price purchase lines.
3. Exclude weekly likelihood observations with incompatible menus or instruments.
4. Keep households with at least two remaining usable weeks.

A week excluded at step 3 can affect later histories. Rebuilding histories after that exclusion would erase a purchase that the researcher actually observed. However, purchase lines removed before weekly aggregation never enter history.

## 4. The history index, derived and worked through

`add_stocks` works household by household in calendar order. Define within-week package shares

$$x_{ijt} = q_{ijt}/Q_{it}.$$

The own-flavor stock is

$$S_{ijt} = \sum_{s<t,\ s \text{ observed}} x_{ijs}.$$

Every observed purchase week contributes total stock one across flavors, regardless of how many packages it contains. Two berry packages and twenty berry packages each add one to berry stock if berry is the only flavor purchased. There is no depreciation, finite window, or division by panel length.

The recent-other-flavor term is

$$N_{ijt} = \sum_{k \neq j} x_{ik,t-1}, \qquad \Xi_{ijt} = (1-\lambda)N_{ijt} - \lambda S_{ijt}.$$

Here $t-1$ means the immediately preceding calendar week. After a gap, $N = 0$ for every flavor. The cumulative $S$ persists across gaps and years. `--habit-weight` sets $\lambda$, default 0.7. Both terms start at zero on a household's first observed week. Current purchases update the stock only after the current index is recorded.

Consider this history in the code's flavor order:

| Week | Current counts | Pre-choice stock S | Recent-other N | Xi at lambda=.7 |
|---|---|---|---|---|
| Jan 2 | (0,2,0) | (0,0,0) | (0,0,0) | (0,0,0) |
| Jan 9 | (1,1,0) | (0,1,0) | (1,0,1) | (.3,-.7,.3) |
| Jan 23 | (0,0,3) | (.5,1.5,0) | (0,0,0) | (-.35,-1.05,0) |

Jan 9 adds half a stock unit to other and half to berry. Jan 23 has no recent term because Jan 16 has no observed purchase. Its current plain purchase affects future stock, not its own current Xi.

Positive gamma lowers utility for a flavor with high own stock and raises it for alternatives different from recent purchases. Negative gamma reverses both effects and represents habit/familiarity under this specification. Gamma is allowed to be negative. Novelty does not mean "never tried": it refers to a flavor different from recent purchases.

Only one gamma scales the fixed combination. The code does not independently estimate habit and novelty effects. Unobserved pre-panel history is set to zero; missing purchases also mean missing history. Since stock grows with observed panel length, panel-length and initial-condition sensitivity matter.

# 5. Utility and the four specifications

For an available inside flavor,

$$V_{ijt,r} = \beta_j + \gamma_r \Xi_{ijt} + \alpha_r p_{ijt} + \sigma \hat{\varepsilon}_{ijt}.$$

The final term exists only with price controls. The normalization is $\beta_{other} = 0$, with common coefficients for berry and plain. Conditional choice only identifies relative utilities: adding the same constant to every inside alternative cancels from probabilities.

| Model argument | Price coefficient | History coefficient |
|---|---|---|
| `naive` | Common negative alpha | Zero |
| `homogeneous` | Common negative alpha | Common unrestricted gamma |
| `mixed_naive` | Random negative alpha | Zero |
| `heterogeneous` | Random negative alpha | Random normal gamma |

The heterogeneous specification uses independent standard normal coordinates:

$$\alpha_r = -\exp(\mu_a + \tau_a z_{r,a}), \qquad \gamma_r = \mu_\gamma + \tau_\gamma z_{r,\gamma}.$$

Alpha is always negative. Gamma has support on both signs. Independence is a restriction: correlated price sensitivity and history preferences are not estimated. Beta never varies across draws or households.

`mu_log_minus_alpha` is $\mu_a$, and `sd_log_minus_alpha` is $\tau_a$, not the standard deviation of alpha in dollars. The implied moments are

$$E[\alpha] = -e^{\mu_a + \tau_a^2/2}, \qquad \mathrm{Var}(\alpha) = (e^{\tau_a^2} - 1)e^{2\mu_a + \tau_a^2}.$$

For $\tau_\gamma > 0$, the habit share is $P(\gamma < 0) = \Phi(-\mu_\gamma/\tau_\gamma)$. A negative population mean does not imply every household has negative gamma. These are population type distributions; the code does not output household posterior coefficient estimates.

### The optional constructed outside count

For positive capacity $K$, the code sets $q_{i0t} = \max(K - Q_{it}, 0)$, adds outside utility zero, and estimates a common inside constant. With `--capacity 7`, a three-package week gets outside count four; a nine-package week gets zero. Missing weeks remain absent.

This is a constructed outcome, not measured nonpurchase. Total modeled count is $\max(K, Q_{it})$, and package counts proxy occasions. Changing K changes the outcome and the likelihood, so its NLL cannot be compared directly to capacity zero as if only a parameter restriction changed.

## 6. The likelihood: why integration comes after summing the history

For available set $A_{it}$, the conditional logit probability is

$$P_{ijt,r} = \frac{e^{V_{ijt,r}}}{\sum_{k \in A_{it}} e^{V_{ikt,r}}}.$$

Include outside in this denominator only when enabled. Conditional on a type r, the household's count-logit log likelihood is

$$\ell_{ir} = \sum_t \sum_j q_{ijt} \log P_{ijt,r}.$$

The multinomial coefficient $Q!/\prod_j q_j!$ is omitted because it does not depend on parameters. Thus reported NLL is suitable for optimizing the same outcomes, but is not the fully normalized multinomial log likelihood.

One household has the same latent type in all its weeks. The simulated likelihood is

$$\widehat{L}_i = \frac{1}{R} \sum_{r=1}^{R} e^{\ell_{ir}}, \qquad \log \widehat{L}_i = \mathrm{LSE}_r(\ell_{ir}) - \log R.$$

The sum over weeks is **inside** the integration. Integrating probabilities separately each week would redraw preferences every week and define another model. A household with many repeated berry choices can make a persistent berry- favoring type much more plausible than a sequence of independently redrawn types.

`make_arrays` produces counts of shape W-by-J, prices/Xi/residuals of shape W-by-3, a W-by-3 availability mask, and a length-W dense household index. `np.bincount` aggregates weekly log likelihood contributions to households. Unavailable utilities become minus infinity. Their zero purchase counts are handled explicitly so the code does not multiply zero by minus infinity.

`logsumexp` subtracts the largest exponent internally, avoiding overflow in both choice denominators and household integration. `household_loglike` processes one draw at a time rather than allocating a W-by-R-by-J utility tensor.

`make_draws` requires an even R of at least two. It generates R/2 independent two-dimensional standard normal draws and appends their negatives. These antithetic draws are fixed throughout a fit and shared as quadrature nodes across households. They approximate the population integral; they are not estimated household identities. Nonmixed models use a single zero draw.

## 7. The analytic gradient and numerical optimization

`demand_objective_gradient` returns mean household NLL and its analytic gradient. For one type draw, the weekly utility score for alternative j is

$$e_{ijt,r} = q_{ijt} - Q_{it}^{model} P_{ijt,r}.$$

For a parameter $\theta_k$, multiply this residual by $\partial V_j/\partial\theta_k$, sum alternatives and weeks, then integrate with the household's posterior draw weights:

$$w_{ir} = \frac{e^{\ell_{ir}}}{\sum_a e^{\ell_{ia}}}, \qquad \frac{\partial \log \widehat{L}_i}{\partial \theta_k} = \sum_r w_{ir} \frac{\partial \ell_{ir}}{\partial \theta_k}.$$

This is the score of the finite-draw simulated likelihood, not a finite-difference approximation. The chain rule gives price derivatives $\alpha_r p_j$ for $\mu_a$, $z_{r,a}\alpha_r p_j$ for $\tau_a$, Xi for mean gamma, and $z_{r,\gamma}\Xi$ for gamma SD. Berry/plain intercept scores select their respective residuals. The residual-control coefficient uses the first-stage residual.

The optimizer minimizes average NLL for scale stability. Printed/checkpointed NLL multiplies it by the household count. `L-BFGS-B` uses the supplied gradient, `ftol=1e-10`, `gtol=1e-6`, and default maximum 600 iterations.

Bounds are implementation restrictions:

| Parameter | Bounds | Initial value |
|---|---|---|
| Any `sd_` parameter | [0,3] | .3 |
| `mu_log_minus_alpha` | [-6,3] | 0 |
| All remaining parameters | [-20,20] | 0 |

Starting SDs away from zero avoids a symmetric stationary point. The first start uses this vector; later starts add seeded normal noise with SD .5 and clip to bounds. Default main starts are three. This is local multistart optimization, not a proof of a global optimum. Importantly, selection keeps the smallest objective even if that candidate did not converge; final convergence must be read from `fit.json`. Boundary distance below `1e-4` produces `at_bound=True`.

More draws improve integration accuracy, more starts probe local solutions, and more iterations allow longer searches. They address different problems. More allocated CPUs alone do not parallelize the draw or bootstrap loops.

# 8. The price control, step by step

## 8.1 The instrument actually used

For week t, let $T_t, C_t$ be the sum and count of valid unique quotes over all consumer DMAs and flavors, and $T_{dt}, C_{dt}$ those in consumer DMA d. Then

$$Z_{dt} = \frac{T_t - T_{dt}}{C_t - C_{dt}}.$$

Each quote is deduplicated within DMA by store/week/UPC/flavor/price. The same physical quote can appear in multiple consumer DMAs if shoppers from different DMAs visited that store; this is a property of the merged input. DMA is the consumer's `dma_cd`, not automatically the retail store's DMA.

Z pools all flavors, weights quotes equally, and is copied into `iv_other`, `iv_berry`, and `iv_plain`. It is not a mean of DMA means or a flavor-specific UPC-matched instrument. A denominator of zero yields missing Z and excludes the observation after history construction.

`price_control.other_dma_instruments` implements a different, UPC-matched retail-DMA construction retained in the module. It is **not called by the current `prepare_data` path**. Its presence does not describe the active instrument.

## 8.2 First-stage regressions

For each flavor j, `add_price_controls` estimates on usable observations with that flavor available:

$$p_{ijt} = \pi_j Z_{dt} + b_j' \Xi_{it} + \eta_{jt} + \kappa_{jd} + \varepsilon_{ijt}.$$

The three Xi columns all enter each equation. There are flavor-specific week and DMA fixed effects. Every household-week gets equal weight; a shared menu price can repeat across many households. Availability creates different samples for the three equations.

`absorb_effects` repeatedly subtracts group means for week and DMA from the matrix containing y, Z, and Xi. In an unbalanced panel, one pass generally reintroduces means for the other effect. The code cycles until the largest change is below `1e-10`, with at most 2,000 cycles.

After absorption, the Frisch–Waugh–Lovell calculation projects Z and y onto Xi, then estimates $\hat{\pi} = (Z_\perp' y_\perp)/(Z_\perp' Z_\perp)$. Least squares tolerates redundant Xi columns. An excluded instrument with no remaining variation stops the fit. An essentially zero residual also stops it.

The resulting residual is in dollars and enters utility with one common unrestricted `price_control` coefficient. An unavailable flavor gets placeholder residual zero and is masked out anyway. No outside residual is constructed.

### 8.3 Diagnostics and identification

Household-cluster scores for the IV slope are $g_i = \sum_{t \in i} Z_{\perp,it} \hat{\varepsilon}_{it}$. The reported variance uses $\sum_i g_i^2/(Z_\perp' Z_\perp)^2$, multiplied by H/(H-1) when H>1. The squared clustered t statistic supplies `iv_cluster_F`; `partial_R2` compares the unrestricted residual sum of squares to y after Xi partialling.

These describe relevance conditional on the included controls. They do not establish exclusion. Common demand shocks, national promotions, and cross-market pricing policies can connect other-DMA prices to local unobserved demand. The linear residual control also imposes a functional-form restriction. Disabling controls with `--no-price-control` removes the IV requirement, first stages, and utility residual term, potentially changing the sample as well as the model.

## 9. Household bootstrap and its scope

`resample_households` samples H household histories with replacement. Selecting the same original household twice creates two new IDs, preserving two independent bootstrap contributions. All retained observations and precomputed Xi travel with the history. Xi is not recomputed from the reduced likelihood sample.

Each replication then refits all enabled price equations and reoptimizes demand. The main estimate supplies the initial vector, with default two local starts. The integration seed and nodes remain fixed, as do constructed instruments and the underlying retail quote universe. Thus this is two-stage household sampling inference conditional on the observed prices/instruments and integration scheme. It does not estimate uncertainty from common DMA/time shocks or simulated integration.

Only converged, finite bootstrap fits enter inference. Expected identification or linear-algebra failures are recorded. After every replication the complete record list is checkpointed to `bootstrap_draws.csv`.

Let B be requested repetitions and V successful fits. Reporting requires $V \geq \max(20, \lceil .8B \rceil)$. For B=49 this means at least 40 successful fits. Reported SEs are sample standard deviations; limits are empirical 2.5 and 97.5 percentiles. Covariance uses the same accepted vectors. Missing SEs mean disabled, skipped, or insufficient inference, not zero uncertainty.

Main nonconvergence skips the bootstrap entirely. Boundary SDs and selected-out failed repetitions complicate interpretation even if the numerical threshold passes. A bootstrap cannot repair misspecified histories or an invalid instrument.

## 10. Running and reading the results

From `final_draft/est_code`, use the existing environment with dependencies from `requirements.txt`. For a development run:

```
python3 est.py --sample /path/to/summary_outputs/yogurt_estimation_sample.parquet \
  --max-households 100 --draws 32 --starts 1 --se-method none \
  --output /path/to/results/development
```

For a larger run, choose a separate output directory:

```
python3 est_het_gamma.py --sample /path/to/summary_outputs/yogurt_estimation_sample.parquet \
  --draws 128 --starts 5 --bootstrap-reps 49 \
  --output /path/to/results/heterogeneous
```

Forty-nine is the current bootstrap default, not an assertion that it suffices for final inference. Increasing to 499 improves tail resolution if computation and fit stability permit. `--seed` defaults to 306, years to 2006–2015, and `--habit-weight` to .7. Run `python3 est.py --help` for the complete interface.

| Output | How to use it |
|---|---|
| `weekly_inputs.csv` | Audit actual counts, prices, availability, histories, instruments, residuals |
| `first_stage.csv` | Flavor-specific IV slope, clustered SE/F, partial $R^2$, residual SD, sample sizes |
| `starts_checkpoint.json` | Completed starts, parameter vectors, NLL, convergence, iteration counts |
| `point_estimates.csv` | Main fit saved before the potentially long bootstrap |
| `gamma_distribution.pdf` | Fitted population gamma distribution, with provisional-fit marking |
| `bootstrap_draws.csv` | Successes, failures, messages, and coefficients after every replication |
| `covariance.csv` | Accepted bootstrap covariance, when reporting criteria pass |
| `estimates.csv` | Estimates, boundary flags, SEs, percentile limits |
| `fit.json` | Final convergence, optimizer message, settings, NLL, starts, inference status |

Repeated use of an output directory can leave old artifacts from another model or from previously enabled inference. Prefer a fresh directory per specification. Checkpoint files record progress; the code does not implement automatic resume.

Inspect in dependency order. First verify sample and menu construction. Then check instrument variation and residual magnitudes. Then compare starts and boundary flags. Next increase draws and change seeds to assess integration sensitivity. Only then interpret bootstrap uncertainty. Compare NLL across models only with identical households, weeks, availability, capacity, and counts.

For synthetic checks without Nielsen data:

```
python3 -m unittest -v test_est.py test_price_control.py test_summary_stats.py
```

## Modifying the estimator carefully

Adding a coefficient involves `parameter_names`, utility construction in both likelihood paths, the analytic score, and appropriate bounds/initialization. Changing only `household_loglike` leaves optimization on a different model, because it uses `demand_objective_gradient`. Changing the flavor set also touches fixed array order, reference labels, price controls, output columns, and tests. Changing histories must preserve the timing convention and the distinction between observation exclusions and information available for later states.

Finally, these estimates do not feed automatically into supply. Their alpha is attached to merged package prices, gamma multiplies cumulative Xi, and beta is relative to other flavor. Supply's fixed-type binary switching parameters need a separate calibration or an explicitly redesigned empirical model.